

Semantics

Let the sets below denote the domains of their respective terms on the meta level.

$$\begin{aligned}
 a, a', \dots &\in \mathbf{Aexp} \\
 b, \dots &\in \mathbf{Bexp} \\
 x, y, z, p, \dots &\in \mathbf{Var} \\
 v, \dots &\in \mathbf{Val} = \mathbb{Z} \\
 n, \dots &\in \mathbf{Numeral} \\
 s, \dots &\in \mathbf{Stm} \\
 \sigma, \sigma', \dots &\in \mathbf{State} \\
 P, \dots &\in \mathbf{Proc} = \mathbf{Var}^* \times \mathbf{Var}^* \times \mathbf{Stm} \\
 \oplus &\in \{+, -, *, /, \%\} \\
 \odot &\in \{\text{not}, \text{and}, \text{or}\} \\
 \ominus &\in \{:=, +=, -=, *=, /=, \%= \} \\
 \otimes &\in \{=, \#, <, <=, >, >= \}
 \end{aligned}$$

For **IMP + EXT**, the state domain is defined as triple of variables, procedures, and break flag. Exceptions are denoted with $\perp$ and some integer status code or irrecoverable failure symbol.

$$\mathbf{State} = (\mathbf{Var} \rightarrow \mathbf{Val}) \times (\mathbf{Var} \rightarrow \mathbf{Proc}) \times \{\text{true}, \text{false}\} \cup \{\perp\} \times (\mathbb{Z} \cup \{!\})$$

The following semantic functions map syntactic representations to their semantic entities. Un-circled operators denote the respective arithmetic or boolean operations or relations.

$$\begin{aligned}
 \mathcal{N}[\![\cdot]\!] &: \mathbf{Numeral} \rightarrow \mathbf{Val} \\
 \mathcal{A}[\![\cdot]\!]_{\sigma} &: \mathbf{Aexp} \rightarrow \mathbf{State} \rightarrow \mathbf{Val} \\
 \mathcal{B}[\![\cdot]\!]_{\sigma} &: \mathbf{Bexp} \rightarrow \mathbf{State} \rightarrow \{\text{true}, \text{false}\}
 \end{aligned}$$

$\mathcal{N}$ maps a numeral to its corresponding integer value, i.e. $\mathcal{N}[\![42]\!] = 42$. Its definition is omitted.

$\mathcal{A}$ is defined inductively and maps an arithmetic expression in some state to an integer value.

$$\begin{aligned}
 \mathcal{A}[\![n]\!]_{\sigma} &= \mathcal{N}[\![n]\!] \\
 \mathcal{A}[\![x]\!]_{\sigma} &= \sigma(x) \\
 \mathcal{A}[\![a_1 \oplus a_2]\!]_{\sigma} &= \mathcal{A}[\![a_1]\!]_{\sigma} * \mathcal{A}[\![a_2]\!]_{\sigma}
 \end{aligned}$$

$\mathcal{B}$ is defined inductively and maps a boolean expression in some state to a truth value.

$$\begin{aligned}
 \mathcal{B}[\![\text{true}]\!]_{\sigma} &= \text{true} \\
 \mathcal{B}[\![\text{false}]\!]_{\sigma} &= \text{false} \\
 \mathcal{B}[\![\text{not } b]\!]_{\sigma} &= \neg \mathcal{B}[\![b]\!]_{\sigma} \\
 \mathcal{B}[\![b_1 \odot b_2]\!]_{\sigma} &= \mathcal{B}[\![b_1]\!]_{\sigma} \circ \mathcal{B}[\![b_2]\!]_{\sigma} \\
 \mathcal{B}[\![a_1 \otimes a_2]\!]_{\sigma} &= \mathcal{A}[\![a_1]\!]_{\sigma} < \mathcal{A}[\![a_2]\!]_{\sigma}
 \end{aligned}$$

Furthermore, the following functions allow for state updates.

$$\begin{aligned}
 _[\![\cdot \mapsto \cdot]\!] &: \mathbf{State} \rightarrow \mathbf{Var} \rightarrow \mathbf{Val} \rightarrow \mathbf{State} \\
 _[\![\cdot \mapsto \cdot]\!] &: \mathbf{State} \rightarrow \mathbf{Var} \rightarrow \mathbf{Proc} \rightarrow \mathbf{State} \\
 _[\![\text{break} = \cdot]\!] &: \mathbf{State} \rightarrow \{\text{true}, \text{false}\} \rightarrow \mathbf{State}
 \end{aligned}$$

Free Variables

$$\text{fv} : \mathbf{Aexp} \cup \mathbf{Bexp} \cup \mathbf{Stm} \rightarrow \mathcal{P}(\mathbf{Var})$$

fv is defined inductively and yields the set of free variables occurring within some construct.

$$\text{fv}(n) = \emptyset$$

$$\text{fv}(x) = \{x\}$$

$$\text{fv}(a_1 \oplus a_2) = \text{fv}(a_1) \cup \text{fv}(a_2)$$

$$\text{fv}(\text{time } s) = \text{fv}(s)$$

$$\text{fv}(\text{true}) = \emptyset$$

$$\text{fv}(\text{false}) = \emptyset$$

$$\text{fv}(\text{not } b) = \text{fv}(b)$$

$$\text{fv}(b_1 \odot b_2) = \text{fv}(b_1) \cup \text{fv}(b_2)$$

$$\text{fv}(a_1 \otimes a_2) = \text{fv}(a_1) \cup \text{fv}(a_2)$$

$$\text{fv}(\text{skip}) = \emptyset$$

$$\text{fv}(x \ominus a) = \{x\} \cup \text{fv}(a)$$

$$\text{fv}(s_1; s_2) = \text{fv}(s_1) \cup \text{fv}(s_2)$$

$$\text{fv}(\text{if } b \text{ then } s_1 \text{ else } s_2 \text{ end}) = \text{fv}(b) \cup \text{fv}(s_1) \cup \text{fv}(s_2)$$

$$\text{fv}(\text{while } b \text{ do } s \text{ end}) = \text{fv}(b) \cup \text{fv}(s)$$

$$\text{fv}(\text{print } a) = \text{fv}(a)$$

$$\text{fv}(\text{read } x) = \{x\}$$

$$\text{fv}(\text{let } x := a \text{ in } s \text{ end}) = \{x\} \cup \text{fv}(a) \cup \text{fv}(s)$$

$$\text{fv}(s_1 \parallel s_2) = \text{fv}(s_1) \cup \text{fv}(s_2)$$

$$\text{fv}(s_1 [] s_2) = \text{fv}(s_1) \cup \text{fv}(s_2)$$

$$\text{fv}(\text{procedure } p(\vec{x}; \vec{y}) \text{ begin } s \text{ end}) = \{p\} \cup \{\vec{x}\} \cup \{\vec{y}\} \cup \text{fv}(s)$$

$$\text{fv}(p(\vec{a}; \vec{z})) = \{p\} \cup \{\vec{z}\} \cup \{\text{fv}(a_i) \mid i \in [m]\}$$

$$\text{fv}(\text{break}) = \emptyset$$

$$\text{fv}(\text{revert } s \text{ if } b \text{ end}) = \text{fv}(s) \cup \text{fv}(b)$$

$$\text{fv}(\text{match } a \text{ with } \overline{v_i : s_i}, \text{ default: } d \text{ end}) = \text{fv}(a) \cup \text{fv}(d) \cup \{\text{fv}(s_i) \mid i \in [m]\}$$

$$\text{fv}(\text{havoc } x) = \{x\}$$

$$\text{fv}(\text{assert } b) = \text{fv}(b)$$

$$\text{fv}(\text{flip}(n) \ s_1 \text{ flop } s_2 \text{ end}) = \text{fv}(s_1) \cup \text{fv}(s_2)$$

$$\text{fv}(\text{raise } a) = \text{fv}(a)$$

$$\text{fv}(\text{try } s_1 \text{ catch } x \text{ in } s_2 \text{ end}) = \{x\} \cup \text{fv}(s_1) \cup \text{fv}(s_2)$$

$$\text{fv}(\text{swap } x \ y) = \{x, y\}$$

$$\text{fv}(\text{timeout } s \text{ after } a \text{ end}) = \text{fv}(s) \cup \text{fv}(a)$$

$$\text{fv}(\text{alternate } s_1 \ s_2) = \text{fv}(s_1) \cup \text{fv}(s_2)$$

Natural Semantics

IMP

The inference rules of standard **IMP** with addition of print and read to facilitate IO.

$$\begin{array}{c}
 \frac{}{\langle \text{skip}, \sigma \rangle \rightarrow \sigma} \text{Skip} \qquad \frac{}{\langle x \ominus a, \sigma \rangle \rightarrow \sigma[x \mapsto \mathcal{A}[[a']]\sigma]} \text{Assign}^1 \\
 \\
 \frac{}{\langle \text{print } a, \sigma \rangle \rightarrow \sigma} \text{Print}^2 \qquad \frac{}{\langle \text{read } x, \sigma \rangle \rightarrow \sigma[x \mapsto v]} \text{Read}^3 \\
 \\
 \frac{\langle s_1, \sigma \rangle \rightarrow \sigma' \quad \langle s_2, \sigma' \rangle \rightarrow \sigma''}{\langle s_1; s_2, \sigma \rangle \rightarrow \sigma''} \text{Sequence} \\
 \\
 \frac{\langle s_1, \sigma \rangle \rightarrow \sigma_1}{\langle \text{if } b \text{ then } s_1 \text{ else } s_2 \text{ end}, \sigma \rangle \rightarrow \sigma_1} \text{If}^4 \qquad \frac{\langle s_2, \sigma \rangle \rightarrow \sigma_2}{\langle \text{if } b \text{ then } s_1 \text{ else } s_2 \text{ end}, \sigma \rangle \rightarrow \sigma_2} \text{Else}^5 \\
 \\
 \frac{\langle s, \sigma \rangle \rightarrow \sigma' \quad \langle \text{while } b \text{ do } s \text{ end}, \sigma' \rangle \rightarrow \sigma''}{\langle \text{while } b \text{ do } s \text{ end}, \sigma \rangle \rightarrow \sigma''} \text{While}^6 \\
 \\
 \frac{}{\langle \text{while } b \text{ do } s \text{ end}, \sigma \rangle \rightarrow \sigma'} \text{End}^7
 \end{array}$$

¹ $a' = \begin{cases} x \otimes a, & \ominus \neq := \\ a, & \text{otherwise} \end{cases}$, where $\otimes$ denotes definition operator's corresponding arithmetic operator

² $\mathcal{A}[[a]]\sigma$ integer output

³ v integer input

⁴ $\mathcal{B}[[b]]\sigma = \text{true}$

⁵ $\mathcal{B}[[b]]\sigma = \text{false}$

⁶ $\mathcal{B}[[b]]\sigma = \text{true}$ and $\sigma(\text{break}) = \text{false}$

⁷ $\sigma' = \begin{cases} \sigma[\text{break} = \text{false}], & \sigma(\text{break}) = \mathcal{B}[[b]]\sigma = \text{true} \\ \sigma, & \text{otherwise} \end{cases}$

EXT

Here, various rules for extensions to **IMP** are given. These three rules are pretty straightforward.

$$\frac{\langle s, \sigma[x \mapsto \mathcal{A}[[a]]\sigma] \rangle \rightarrow \sigma'}{\langle \text{let } x := a \text{ in } s \text{ end}, \sigma \rangle \rightarrow \sigma'[x \mapsto \sigma(x)]} \text{Local}$$

$$\frac{\langle s_1, \sigma \rangle \rightarrow \sigma_1 \quad \langle s_2, \sigma \rangle \rightarrow \sigma_2}{\langle s_1 \parallel s_2, \sigma \rangle \rightarrow \sigma_1 \cup \sigma_2} \text{Parallel}^8$$

$$\frac{\langle s_1, \sigma \rangle \rightarrow \sigma_1 \quad \langle s_2, \sigma \rangle \rightarrow \sigma_2}{\langle s_1 [] s_2, \sigma \rangle \rightarrow \sigma' \in \{\sigma_1, \sigma_2\}} \text{Nondeterminate}$$

Procedures require some finesse and tweaking the state domain definition.

$$\frac{}{\langle \text{procedure } p(\vec{x}; \vec{y}) \text{ begin } s \text{ end}, \sigma \rangle \rightarrow \sigma[p \mapsto (\vec{x}, \vec{y}, s)]} \text{Procedure}$$

$$\frac{\langle s, \sigma[x_i \mapsto \mathcal{A}[[a_i]]\sigma] \rangle \rightarrow \sigma'}{\langle p(\vec{a}; \vec{z}), \sigma \rangle \rightarrow \sigma[z_j \mapsto \sigma'(y_j)]} \text{Invocation}^9$$

Exception handling can also be achieved, where $\perp[v]$ denotes an exception with value v .

$$\frac{}{\langle \text{raise } a, \sigma \rangle \rightarrow \perp[\mathcal{A}[[a]]\sigma]} \text{Raise}$$

$$\frac{\langle s_1, \sigma \rangle \rightarrow \sigma_1}{\langle \text{try } s_1 \text{ catch } x \text{ in } s_2 \text{ end}, \sigma \rangle \rightarrow \sigma_1} \text{Try}$$

$$\frac{\langle s_1, \sigma \rangle \rightarrow \perp[v] \quad \langle s_2, \sigma[x \mapsto v] \rangle \rightarrow \sigma_2}{\langle \text{try } s_1 \text{ catch } x \text{ in } s_2 \text{ end}, \sigma \rangle \rightarrow \sigma_2} \text{Catch}$$

Transactional executions, loop breaks, assertions, random and swap assignments are also possible.

$$\frac{\langle s, \sigma \rangle \rightarrow \sigma'}{\langle \text{revert } s \text{ if } b \text{ end}, \sigma \rangle \rightarrow \sigma} \text{Revert}^{10}$$

$$\frac{}{\langle \text{break}, \sigma \rangle \rightarrow \sigma[\text{break} = \text{true}]} \text{Break}$$

$$\frac{}{\langle \text{assert } b, \sigma \rangle \rightarrow \sigma} \text{Pass}^{11}$$

$$\frac{}{\langle \text{assert } b, \sigma \rangle \rightarrow \perp[!]} \text{Fail}^{12}$$

$$\frac{}{\langle \text{havoc } x, \sigma \rangle \rightarrow \sigma[x \mapsto n]} \text{Havoc}$$

$$\frac{}{\langle \text{swap } x \ y, \sigma \rangle \rightarrow \sigma[x \mapsto \sigma(y), y \mapsto \sigma(x)]} \text{Swap}$$

⁸ $\sigma_1 \cup \sigma_2$ denotes combination of both states where the left operand dominates in conflicts

⁹ $\sigma(p) = (\vec{x}, \vec{y}, s)$ such that $|\vec{x}| = |\vec{a}|$ and $|\vec{y}| = |\vec{z}|$

¹⁰ $\mathcal{B}[[b]]\sigma' = \text{true}$

¹¹ $\mathcal{B}[[b]]\sigma = \text{true}$

¹² $\mathcal{B}[[b]]\sigma = \text{false}$

There's more than one way to loop. Note that `_times` is only internally accessible.

$$\frac{\langle s; \text{while not } b \text{ do } s \text{ end}, \sigma \rangle \rightarrow \sigma'}{\langle \text{do } s \text{ until } b \text{ end}, \sigma \rangle \rightarrow \sigma'} \text{Do}$$

$$\frac{\langle \text{let } x := a_1 \text{ in while } x < a_2 \text{ do } s; x += 1 \text{ end end}, \sigma \rangle \rightarrow \sigma'}{\langle \text{for } x := a_1 \text{ to } a_2 \text{ do } s \text{ end}, \sigma \rangle \rightarrow \sigma'} \text{For}$$

$$\frac{\langle \text{for } _times := 0 \text{ to } n \text{ do } s \text{ end}, \sigma \rangle \rightarrow \sigma'}{\langle \text{repeat } a \text{ times } s \text{ end}, \sigma \rangle \rightarrow \sigma'} \text{Repeat}^{13}$$

The flip construct alternates execution between branches, where `_flip_n` is only used internally. Zero or more cases and one default can be matched on.

$$\frac{\langle s_1, \sigma \rangle \rightarrow \sigma_1}{\langle \text{flip}(n) \ s_1 \ \text{flop } s_2 \text{ end}, \sigma \rangle \rightarrow \sigma_1[_{\text{flip}_n} \mapsto 1]} \text{Flip}^{14}$$

$$\frac{\langle s_2, \sigma \rangle \rightarrow \sigma_2}{\langle \text{flip}(n) \ s_1 \ \text{flop } s_2 \text{ end}, \sigma \rangle \rightarrow \sigma_2[_{\text{flip}_n} \mapsto 0]} \text{Flop}^{15}$$

$$\frac{\langle s_k, \sigma \rangle \rightarrow \sigma'}{\langle \text{match } a \text{ on } \overline{v_i : s_i}, \text{ default: } d \text{ end}, \sigma \rangle \rightarrow \sigma'} \text{Case}^{16}$$

$$\frac{\langle d, \sigma \rangle \rightarrow \sigma'}{\langle \text{match } a \text{ on } \overline{v_i : s_i}, \text{ default: } d \text{ end}, \sigma \rangle \rightarrow \sigma'} \text{Default}^{17}$$

¹³ $\mathcal{A}[[a]]\sigma = n$

¹⁴ $\mathcal{A}[_{\text{flip}_n}]\sigma = 0$

¹⁵ $\mathcal{A}[_{\text{flip}_n}]\sigma = 1$

¹⁶ $\mathcal{A}[[a]]\sigma = v_k$

¹⁷ $\mathcal{A}[[a]]\sigma \notin \{v_i \mid i \in [m]\}$